Predicting 10% stock gains within 30 days

An end-to-end supervised machine learning workflow

Class project
577,400 stock-day snapshots · 87 features · 2015–2025

Executive summary

This project builds a binary classification model that predicts whether a stock will gain at least 10% within 30 trading days from a given snapshot. The dataset contains 577,400 snapshots across 574 stocks from 2015 through 2025, with 87 pre-engineered technical-indicator features. The target class is imbalanced, with 10.4% positives.

We followed the canonical Supervised Learning Workflow from course deck #17, then added empirical experiments that the deck recommends but that students often skip: a skewness audit and log-transform comparison, an imbalance-treatment comparison (ROS / RUS / SMOTE versus class weighting), a five-model comparison across Logistic Regression / Random Forest / LightGBM / Linear SVM / KNN, threshold-aware operating-point analysis, and a feature-engineering pass that added interaction, symbol-level historical, lag, and time-of-year features.

The headline finding is that the simplest model — a class-weighted Logistic Regression with L2 regularization — turned out to be the strongest. Random Forest and a tuned LightGBM converged to within 0.02 of LR on every threshold-independent metric (PR-AUC and ROC-AUC). KNN underperformed substantially due to the curse of dimensionality. We call this the linear ceiling: with the current features, the signal in this dataset is largely linear and additive, and adding model complexity does not help. The bottleneck for further improvement is feature engineering (or a re-framing of the problem), not model choice.

After adding 14 engineered features, the LR model achieves on the held-out 2024–2025 test set: F1 0.536, PR-AUC 0.479, ROC-AUC 0.902, with precision 0.467 and recall 0.630 at the validation-chosen threshold of 0.741. At the high-precision operating point (threshold 0.986), the model identifies approximately 1–2 stocks per trading day with 64% probability of gaining 10% — a 5.8x lift over the 11% random baseline. Above 70% precision is not achievable with the current features; this is the model's hard ceiling.

1. Introduction

1.1 Business question

Given a snapshot of a stock at a point in time — represented as a feature vector of technical indicators and pattern-detection flags — can we predict whether that stock will gain at least 10% within the next 30 trading days?

1.2 Technical (machine-learning) objective

Binary classification of tabular numeric and categorical features. The output is a probability that gain_10pct_30d is True; the operating threshold is chosen based on the precision-recall trade-off appropriate to the downstream use case.

1.3 Success metric

Because the classes are imbalanced (about 10% positive), accuracy is misleading: a model that always predicts False would achieve 89.6% accuracy and be useless. We optimize PR-AUC (the area under the precision-recall curve) and F1 score, and we report the precision-recall trade-off at a chosen threshold. ROC-AUC is reported for completeness but is known to be flattering under class imbalance.

2. Data

2.1 Source and structure

Pre-breakout filtered stock snapshots, 2015-01-02 through 2025-12-31, covering 574 unique symbols across NYSE, NYSE ARCA, and NASDAQ. The total row count is 577,400. The target column gain_10pct_30d was computed by an upstream pipeline that examines the next 30 trading days of close prices.

2.2 Feature taxonomy

The 87 columns break down as follows:

- Identifiers (3): symbol, exchange, timestamp.
- Numeric features (45): price-derived indicators including RSI(14), MACD and histogram, Stochastic %K, ADX with +DI / -DI, ATR(5) and ATR(10), ADR (average daily range), Bollinger Band width %, OBV, CCI(20); volume statistics (raw volume, 30-day average, volume ratio); pattern-detection scalars (VCP touch count, consolidation range %, T1 and T2 depths); positional metrics (distance from all-time high, distance from 200-day MA, days since ATH, MA slope); pattern-action scoring (body, momentum, close, volume sub-scores).
- Boolean flags (24): pattern detection (breakout_detected, breakout_is_strong / weak / suspicious, vcp_is_valid, ma_alignment, above_fast_ma, above_long_ma) and warning flags.

- Categorical labels (16): pa_quality_label, weekly_trend_status, market_regime, vcp_state, slope_category, volume_surge_label, rs_state, vcs_label, macd_state, stoch_state, adx_state, cci_state, obv_state, rsi_pullback_label, signal_type.
- Target (1): gain_10pct_30d, boolean.

2.3 Feature engineering already done upstream

The dataset arrives with feature engineering complete: raw OHLCV series have been transformed into the technical indicators above using standard formulae from technical analysis (Wilder smoothing for RSI and ADX, EMAs for MACD, SMAs for the long-term trend reference). No additional feature derivation was needed before modeling.

3. Exploratory data analysis

3.1 Class balance

The positive class accounts for 10.38% of the dataset (59,950 of 577,400 rows). This is roughly a 1:9 imbalance, which informs every downstream choice: it rules out accuracy as a primary metric, it triggers imbalance-aware techniques, and it makes PR-AUC the most informative summary statistic.

3.2 Temporal structure and regime drift

The positive rate varies substantially year-to-year, ranging from 4.3% in 2017 to 22.0% in 2020. This is not noise — it reflects market regime. Periods like 2020 (COVID rebound) and 2022 (volatility expansion) saw a much higher rate of 10% pops than calm bull-market periods. The dataset includes a market_regime feature (Bullish / Bearish / Neutral) that captures part of this signal: positive rate is 21.1% in Bearish months, 9.6% in Neutral, and 8.0% in Bullish.

3.3 Missing values

There are zero missing values in the dataset — but not because the data was inherently complete. The upstream data pipeline already filtered out rows with bad or absent data. Two specific kinds of rows were removed before we received the dataset: (1) inactive instruments — ETFs and stocks that were initially traded but later went dormant, where zero trading volume causes ATR, ADR, RSI and Bollinger-band-width calculations to become undefined or stale; and (2) API-corrupted snapshots with missing OHLC fields or corrupted volume readings. The course's missing-data treatment material does not need to be applied here, but this filtering introduces a mild survivorship-style bias that we explicitly flag in the limitations section.

3.4 Strongest single-feature signals

Linear correlation with the target. The strongest positive correlations are with volatility and consolidation features: ADR (+0.47), Bollinger Band width % (+0.46), past ADR (+0.42), consolidation range % (+0.41), distance from all-time high (+0.39), and the VCP T1 and T2 depth metrics (+0.36 to

+0.38). Strongest negative correlations are with above-long-MA flags (around −0.25). Categorical features show meaningful lift in specific values: slope_category 'strong' has a 44.0% positive rate vs 5.2% for 'weak', and vcp_state 'Failed Breakout' has a 35.7% positive rate.

3.5 Multicollinearity and skewness audit

The volatility cluster (ADR, BB width %, consolidation range %, VCP depths) shows pairwise correlations of 0.6–0.92. We dropped one of two perfectly anti-correlated MA flags (warning_below_long_ma vs above_long_ma at −0.97), and during preprocessing also dropped atr_5, avg_volume_30d, adr_past, and minus_di to reduce multicollinearity that hurts logistic regression coefficients.

A skewness audit found 19 of 45 numeric features with |skew| > 2 on the training set, and 12 of them with |skew| > 5. Worst offenders included atr_5 (+71), atr_10 (+62), macd_histogram (−25), volume (+8), and adr (+5). We tested the deck #17 prescription of log/signed-log transformations on these features. Result: LR validation F1 went from 0.527 to 0.520 and PR-AUC from 0.469 to 0.470 — essentially no change. Capping at the 1st/99th percentile combined with standardization was already enough; the additional log transform was not needed for this dataset and this model. The transformation is implemented in the notebook as an opt-in flag (APPLY_LOG_TRANSFORM = True) for completeness.

4. Methodology

4.1 Time-based split with 30-day embargo

The dataset is fundamentally a time series: each row's label depends on the next 30 days of price history. A random train/test split would let snapshots from adjacent dates appear in both training and test sets, leaking information from the future. We split chronologically:

- Train: 2015-01-02 through 2022-12-01 (366,176 rows; positive rate 10.12%).
- Validation: 2023-01-03 through 2023-12-01 (55,558 rows; positive rate 10.41%).
- Test: 2024-01-02 through 2025-12-31 (145,284 rows; positive rate 11.07%).

Because each row's target is a 30-day forward window, we leave a 30-day embargo gap between train end and validation start, and another between validation end and test start — verified at 33 and 32 calendar days respectively. The class balance is consistent across all three splits, indicating that the split did not accidentally over-sample any particular regime.

4.2 Preprocessing pipeline

4.2.1 Drops

- Identifier columns: symbol, exchange, timestamp.
- Constant columns: signal_type (every row equals 'pre_breakout').
- All-False booleans: is_biotech, strong_uptrend.

- Multicollinear features: warning_below_long_ma, atr_5, avg_volume_30d, adr_past, minus_di.

4.2.2 Outlier capping (deck #3)

Each numeric feature is clipped at the 1st and 99th training-set percentiles. This is the 'capping' treatment from deck #3.

4.2.3 Categorical encoding

Five columns have a natural order and are integer-encoded with explicit mappings (pa_quality_label, volume_surge_label, slope_category, adx_state, vcs_label). The remaining nine nominal columns are one-hot encoded. After encoding, the feature matrix has approximately 100 columns.

4.2.4 Standard scaling

Numeric features are z-scored (mean 0, std 1) using training statistics. Required for distance- and weight-based models (logistic regression, KNN, linear SVM). Tree-based models are scale-invariant and operate on the unscaled feature matrix.

4.3 Imbalance treatment empirical comparison (deck #13)

Per deck #13, we compared the class_weight='balanced' baseline against three resampling techniques: ROS (random oversampling), RUS (random undersampling), and SMOTE (synthetic minority oversampling). All sampling techniques are applied to the training set only; validation and test always reflect the real 10% positive rate. The results are summarized in section 5.2.

4.4 Models compared

Five model families are trained and compared on the validation set:

- Logistic regression with class_weight='balanced' and L2 regularization (the baseline).
- Random Forest with class_weight='balanced' (300 estimators, max_depth 12, min_samples_leaf 20).
- LightGBM with scale_pos_weight imbalance handling, num_leaves 31, max_depth 6, regularization (alpha and lambda 0.1), early stopping on average_precision.
- K-nearest neighbors with k=25 and distance weighting, sub-sampled to 50,000 training rows.
- Linear SVM, calibrated to produce probabilities via 3-fold cross-fit isotonic regression.

Tree models train on the unscaled feature matrix; LR, KNN, and SVM train on the scaled version. Each model's threshold is tuned to maximize F1 on the validation set; we then compare both threshold-independent metrics (PR-AUC, ROC-AUC) and the F1 at the chosen threshold.

4.5 Hyperparameter tuning experiment

Two hyperparameter experiments were conducted:

First, on Random Forest: increasing max_depth from 12 to 20 and decreasing min_samples_leaf from 20 to 10 raised F1 at threshold 0.5 from 0.504 to 0.515 — but PR-AUC dropped from 0.468 to 0.455 and ROC-AUC from 0.894 to 0.891. The model was overfitting; F1 at a fixed threshold improved by accident,

not because the underlying model became better. This is the deck #11 cautionary tale about hyperparameter tuning on threshold-dependent metrics.

Second, on LightGBM: the initial out-of-the-box configuration with class_weight='balanced' produced PR-AUC 0.423 — significantly worse than LR. After replacing class_weight with scale_pos_weight (LightGBM's native imbalance handling), reducing num_leaves from 63 to 31, max_depth from 8 to 6, adding L1 and L2 regularization, and switching the early-stopping eval_metric to average_precision, LightGBM recovered to PR-AUC 0.449 — competitive with but not exceeding LR.

4.6 Feature engineering pass

After establishing the linear ceiling across all five model families, we added 14 engineered features to test whether feature engineering rather than model complexity could break the ceiling:

- Interaction features (5): adr × volume_ratio, adr × ath_distance_pct, consolidation_range_pct × adr, atr_10 / close, volume / avg_volume_30d. LR cannot discover interactions on its own; tree models can but it does not seem to help much here.
- Symbol-level historical context (3): symbol_hist_pos_rate (expanding mean of past target labels, shifted by 31 rows to prevent leakage from the 30-day forward window), symbol_adr_60d_mean (60-day rolling mean of ADR, shifted by 1), symbol_rsi_60d_mean (same for RSI).
- Lag and momentum (4): 5-day differences in RSI(14), ADR, BB width %, consolidation range %.
- Time-of-year (3): month, day_of_week, quarter (one-hot encoded after merging).

The 30-day target window means symbol-level historical features need a careful shift: a past target label at row N-1 used prices through 30 days after that row, so to avoid leakage we use only labels from at least 31 rows back. The notebook implements this with df.groupby('symbol')[TARGET].transform(lambda x: x.shift(31).expanding().mean()).

5. Results

5.1 Imbalance-treatment comparison

All four LR variants at threshold 0.5 on validation:

Variant	Precision	Recall	F1	PR-AUC
LR baseline (class_weight)	0.380	0.766	0.508	0.469
LR + ROS	0.351	0.797	0.487	0.461
LR + RUS	0.351	0.798	0.487	0.456
LR + SMOTE	0.348	0.798	0.485	0.456

All three resampling techniques are essentially identical to each other (within 0.001 on every metric) and all three are slightly worse than the simpler class_weight='balanced' baseline. The likely reasons: 10% imbalance is mild rather than extreme; with 366,000 training rows we already have ~37,000

positive examples without synthetic augmentation; and SMOTE struggles with one-hot encoded categoricals because it interpolates between rows producing fractional one-hots that do not match real categories. The lesson from deck #13 is to test these techniques rather than assume they help.

5.2 Five-model comparison on validation

Each model evaluated at its appropriate threshold (the optimal F1 threshold for LR and LightGBM, the default 0.5 for the others):

Model	Threshold	F1	PR-AUC	ROC-AUC	Verdict
Logistic Regression (baseline)	0.63 (opt)	0.527	0.469	0.893	Winner
LightGBM (tuned)	0.31 (opt)	0.525	0.449	0.893	Tied
Random Forest	0.50	0.504	0.468	0.894	Tied
Linear SVM (calibrated)	0.50	0.413	0.462	0.893	Worse F1
KNN (k=25, 50k subsample)	0.50	0.260	0.423	0.855	Significantly worse

Four of five model families converge to PR-AUC 0.45–0.47 and ROC-AUC ≈ 0.89. KNN is the only model meaningfully worse — both because of the curse of dimensionality (with 100+ features after one-hot, Euclidean distance becomes nearly meaningless) and because at k=25 with a 10% positive class, predictions rarely cross the 0.5 threshold, leading to recall of only 17%. SVM also has low recall at threshold 0.5 (0.34) and would benefit from the same threshold optimization we applied to LR and LightGBM.

5.3 The linear-ceiling finding

Three completely different model families — a linear model (Logistic Regression), a bagged ensemble of decision trees (Random Forest), and gradient boosting (LightGBM) — converge to nearly identical performance on every threshold-independent metric. This is what an information ceiling looks like when you observe it empirically.

Three implications follow:

- The signal in this dataset is largely linear and additive. When non-linear models cannot beat a linear one, it usually means the relationships between features and target are roughly monotonic and additive. Whoever built the upstream feature engineering captured most of the non-linear interactions already.
- Adding model complexity does not help. Random Forest with 300 trees and LightGBM with hundreds of boosted rounds both tied LR. The complexity is wasted.

- The bottleneck for further improvement is feature engineering or a re-framing of the problem, not model choice. The simplest model is preferred for production: faster, more interpretable, easier to debug and explain.

5.4 Feature engineering — modest improvement

After adding the 14 engineered features described in §4.6, the LR model's metrics improved modestly:

Metric	Before engineering	After engineering
F1 (val, opt threshold)	0.527	0.536
PR-AUC (val)	0.469	0.479
ROC-AUC (val)	0.893	0.902

The improvement is real but bounded — the ceiling moved up about 1 percentage point of PR-AUC, not a transformation. Feature engineering helps but does not break the fundamental information limit of this dataset.

5.5 Final test-set evaluation

After model selection and feature engineering on validation, the engineered LR was evaluated exactly once on the held-out 2024–2025 test set:

Metric	Test (2024–2025)
Threshold (chosen on validation)	0.741
F1	0.536
Precision	0.467
Recall	0.630
PR-AUC	0.479

ROC-AUC on test is 0.902. Confusion matrix at threshold 0.741: 117,618 true negatives, 11,580 false positives, 5,956 false negatives, 10,130 true positives. The model correctly identifies 63% of real winners and is right 47% of the time when it flags a stock. This is a 4.2x lift over the 11% random baseline.

5.6 Operating-point analysis: trading off precision and recall

Because precision and recall trade off against each other, we audited multiple operating points to characterize the model's behavior across the spectrum:

Threshold	Stocks flagged	Test precision	Test recall	Lift over baseline
0.741 (default)	21,710 (15%)	46.7%	63.0%	4.2x
0.892	13,101 (9%)	50.2%	42.2%	4.6x
0.986 (max precision)	705 (0.5%)	63.7%	2.9%	5.8x

The model has a hard precision ceiling around 64%. Targets above 70% precision are not achievable — even when the model is asked to be 100% certain, validation precision tops out below 70%. This reflects irreducible noise in the data.

The 0.986 operating point is the most actionable for a real user: across two years of test data (about 500 trading days), the model flags 705 stocks (~1–2 per trading day), and ~64% of those flags really do produce 10% gains in 30 days. This is a 5.8x lift over a random baseline of 11%.

6. Interpretability — what the model learned

6.1 Top features by coefficient magnitude

The top 12 features by |coefficient| in the engineered LR (standardized features):

Feature	Coefficient
adr	+1.024
ma_alignment	+0.851
pa_quality_score	−0.851
vcp_state_No Pattern	−0.797
atr_10	+0.679
atr_5	−0.627
breakout_is_suspicious	−0.596
ath_distance_pct	+0.577
breakout_is_weak	−0.567
rsi_pullback_label_healthy	−0.551
above_long_ma	+0.498
ma_200	−0.498

6.2 The story the features tell

Volatility wins. The largest positive coefficient is ADR (+1.02). Combined with ATR_10 (+0.68) and consolidation_range_pct, the message is clear: high-volatility, range-expanding stocks are far more likely to gain 10% in a month than calm ones.

Room to recover. ath_distance_pct (+0.58) — being far below all-time high is a positive signal. Stocks already extended (high ma_200, coefficient −0.50) have less room to pop.

Trend confirmation. ma_alignment (+0.85) and above_long_ma (+0.50) are positive — when the moving averages are bullishly aligned and the stock is above its long MA, it is more likely to pop.

Counterintuitive findings. vcp_state_'Failed Breakout' has a positive coefficient (+0.46). EDA confirmed this is correct: 'Failed Breakout' had a 35.7% positive rate, the highest of any VCP state value. Stocks that fail an initial breakout often re-attempt within the 30-day window.

Multicollinearity caveat. atr_5 (−0.63) and atr_10 (+0.68) are 0.98 correlated. The signs flipping does not mean the volatility signal is contradictory — it means the LR model is splitting credit across these two near-identical features in a way that obscures the individual interpretation while preserving the net

effect. For per-feature interpretation, tree-based feature importance (or SHAP on the LightGBM model) is more reliable than LR coefficients on multicollinear data.

7. Reading the predictions correctly

ROC-AUC of 0.90 does NOT mean the model is right 90% of the time. It means: if you pick a random winner and a random non-winner, the model gives the winner a higher score 90% of the time. It is a ranking quality metric, not an accuracy claim. This distinction is important for any presentation of the model.

In practical terms, if the model flags a stock at the default operating threshold:

- About 47% of flagged stocks will actually pop 10% in 30 days (precision = 0.467).
- The model will catch 63% of all real winners (recall = 0.630), missing 37%.
- Compared to the 11% baseline rate, flagged stocks have a 4.2x lift.

If the user wants higher confidence (the high-precision operating point):

- At threshold 0.986, 64% of flagged stocks will pop — a 5.8x lift.
- But only 705 stocks across two years of test data are flagged, missing 97% of real winners.
- This operating mode is best for users who want fewer high-conviction signals to evaluate.

8. Limitations and disclaimers

- Classification is not a trading strategy. This work outputs a probability per stock-day. A real trading strategy would also need position sizing, transaction costs, slippage, holding-period mechanics, and capital allocation rules. None of those are addressed here.
- The model has a hard precision ceiling around 64%. Above 70% precision is not achievable with the current features. This is irreducible noise, not a tuning failure.
- Concept drift is real. Markets change. Feature relationships that hold in 2015–2022 training data may degrade in unseen regimes. Continuous monitoring would be required in production.
- Pre-filtered universe and survivorship-style bias. Zero missing values does NOT mean clean data. The upstream pipeline removed inactive ETFs and dormant stocks (no trading volume → undefined indicators) AND API-corrupted snapshots. Conclusions apply only to actively-traded stocks that passed both the pre_breakout filter and the data-quality filter. Generalization to the broad market or to dormant tickers is not validated here.
- Symbol-level memorization risk. 574 stocks each appear thousands of times. Our split is by time, not by symbol. A leave-one-symbol-out cross-validation would test whether the model has learned generalizable patterns or has memorized specific tickers.

- Probability calibration. Class-weighted training shifts predicted probabilities upward; the best F1 threshold landed at 0.74, not 0.5. For decisions that depend on calibrated probabilities, a separate calibrator (Platt scaling or isotonic regression) should be fit on a held-out slice.
- Lookahead bias in upstream features. Features like sector_rs and the VCP detection signals were precomputed by an external pipeline. Each one needs to be verified to use only data available at the snapshot date.
- Embargo is partial. We use a 30-day gap at split boundaries. More rigorous protocols (combinatorial purged cross-validation) exist in financial machine-learning literature.

9. Conclusions

Five model families (Logistic Regression, Random Forest, LightGBM, Linear SVM, KNN), four imbalance treatments (class weighting, ROS, RUS, SMOTE), one skewness-correction experiment, two hyperparameter-tuning experiments, and one feature-engineering pass with 14 added features all converged to a single empirical conclusion: the simplest model wins. A class-weighted Logistic Regression with L2 regularization, trained on the engineered feature set, achieves F1 0.536, PR-AUC 0.479, and ROC-AUC 0.902 on the held-out 2024–2025 test set.

The deeper finding is the linear ceiling: with the current features, three completely different model families converge to within 0.02 of each other on every threshold-independent metric. The signal is largely linear, the upstream feature engineering already captured most of the non-linear structure, and the bottleneck for further improvement is feature engineering or a problem re-framing — not model choice. KNN underperformed substantially because of the curse of dimensionality.

The model's most useful operating mode is high-precision screening: at threshold 0.986, it identifies approximately 1–2 stocks per trading day with 64% probability of gaining 10% in the next 30 days, a 5.8x lift over the 11% random baseline. Above 70% precision is not achievable with the current features.

The core lesson from this project for any future ML student: every step of the deck #17 workflow is a hypothesis to test, not a recipe to follow. Skewness correction did not help. ROS / RUS / SMOTE were worse than class weighting. Random Forest did not beat LR. LightGBM did not beat LR. Tuning LR's hyperparameters did not help. Each negative result is a finding worth documenting — they collectively tell the story of where the data's information actually is.

Appendix A — Reproducibility

The accompanying Jupyter notebook (Pre_breakout_ML_project.ipynb) reproduces the full pipeline. Required packages: pandas, numpy, scikit-learn, lightgbm or xgboost, imbalanced-learn (for ROS/RUS/SMOTE), shap, matplotlib, seaborn. Random seed 42 is used throughout. Training the LR baseline takes about 20 seconds on a single CPU core; LightGBM with the search above runs in 5–10 minutes; KNN on a 50,000-row subsample runs in under a minute.

Appendix B — File index

- Pre_breakout_ML_project.ipynb — full reproducible notebook with engineered features and all five models.
- Pre_breakout_ML_project_slides.pptx — class presentation (22 slides).
- Pre_breakout_ML_project_report.docx — this written report.
- figures/ — saved chart images from the LR baseline.